

Introduction to CSS

In the [HTML pack](#), you learned how HTML describes the structure and meaning of a page. HTML gives the browser headings, paragraphs, links, images, forms, and sections to work with.

CSS is the next layer: it tells the browser how that HTML should look, including the font size, text color, background color, spacing, borders, rounded corners, layout, and how the page should adapt to different screen sizes.

HTML and CSS work together, but they do different jobs: HTML says what something is, and CSS says how it should be presented.

html

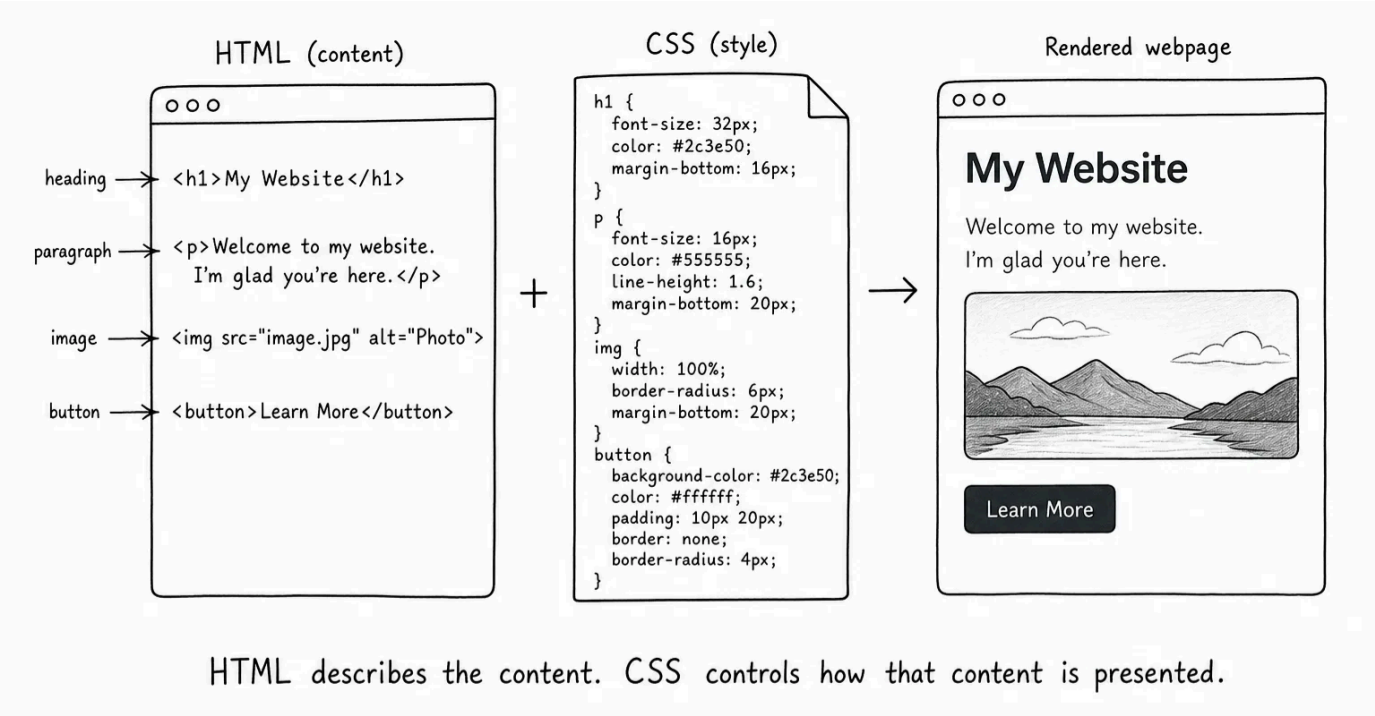
<button>Save changes</button>

That HTML gives the browser a button. CSS can decide whether the button is blue or gray, small or large, square or rounded, flat or shadowed, full-width or only as wide as its text.

CSS

button {
 background-color: #2563eb;
 color: white;
 padding: 12px 16px;
 border: 0;
 border-radius: 6px;
}

CSS lets you take the structure from HTML and turn it into a page that feels designed.



CSS is made of style rules

A CSS file is mostly a list of style rules. Each rule chooses something on the page and gives the browser one or more declarations for how that thing should look.

CSS

```
p {  
  color: #444;  
  font-size: 18px;  
}
```

The rule has three parts you will see everywhere:

- **p** is the **selector**. It chooses which elements the rule applies to.
- **color** and **font-size** are **properties**. They name what you want to change.
- **#444** and **18px** are **values**. They say what each property should

< Previous Lesson

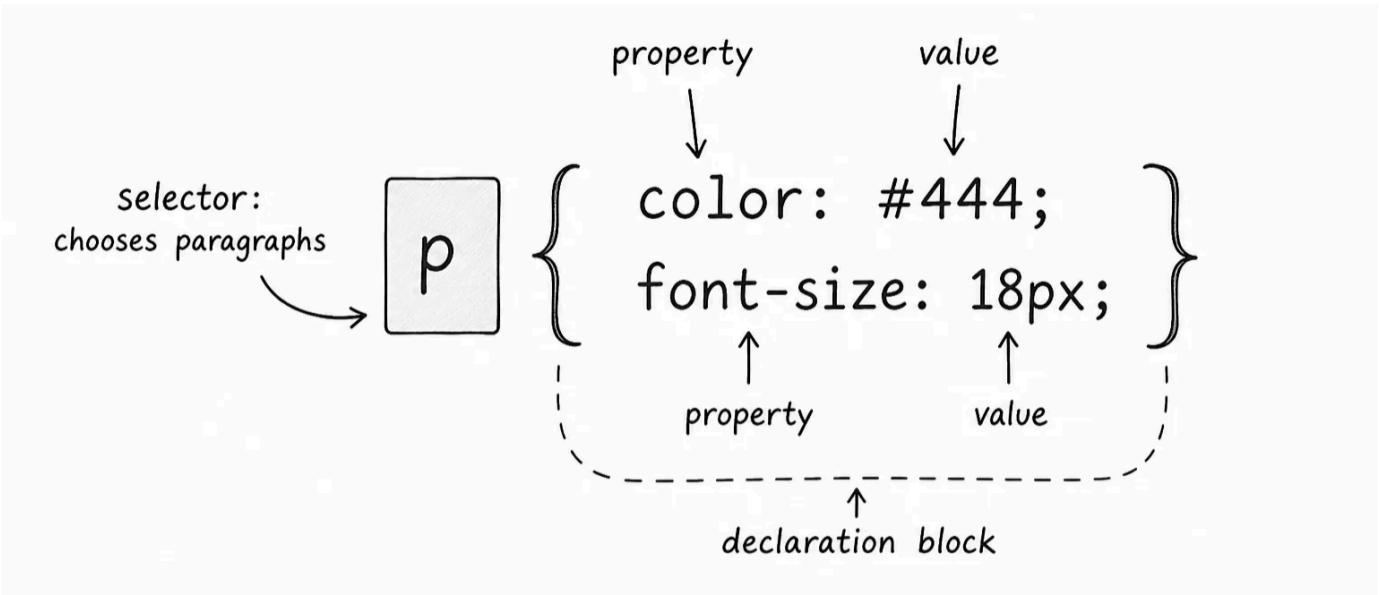
Next Lesson >

The property and value together are called a **declaration**. Each declaration ends with a semicolon, and the declarations sit inside curly braces:

CSS

```
selector {  
  property: value;  
}
```

You do not need to memorize every CSS property before writing CSS. You need to understand that pattern first.



💡 **Read CSS from left to right** - When you see a CSS rule, ask two questions: what does this selector choose, and what declarations does it apply to the matching elements?

Add CSS to HTML

There are three common places you can write CSS.

The smallest option is the **style** attribute. It puts CSS directly on one HTML element:

html

```
<p style="color: #2563eb;">This paragraph is blue.</p>
```

That works, but it mixes the content and the styling in the same line. It also only styles that one element, so it becomes messy if you want several paragraphs to share the same look.

Another option is a **<style>** element inside the **head** :

html

```
<head>
  <style>
    p {
      color: #2563eb;
      font-size: 18px;
    }
  </style>
</head>
```

This keeps the CSS in one place inside the HTML file. It is useful for tiny examples and quick experiments.

For this pack, you will mostly use a separate stylesheet. A stylesheet is a **.css** file linked from your HTML. This is easier to reuse and closer to how real projects organize CSS.

Imagine you have these two files in the same folder:

text

```
index.html
style.css
```

Inside **index.html** , link the stylesheet from the **head** :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My styled page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello, CSS</h1>
    <p>This paragraph will be styled by the CSS file.</p>
  </body>
</html>
```

The `<link>` element tells the browser to load `style.css` and treat it as a stylesheet. The `href` value is the path to the CSS file, just like an image `src` is the path to an image.

Now `style.css` can style the HTML:

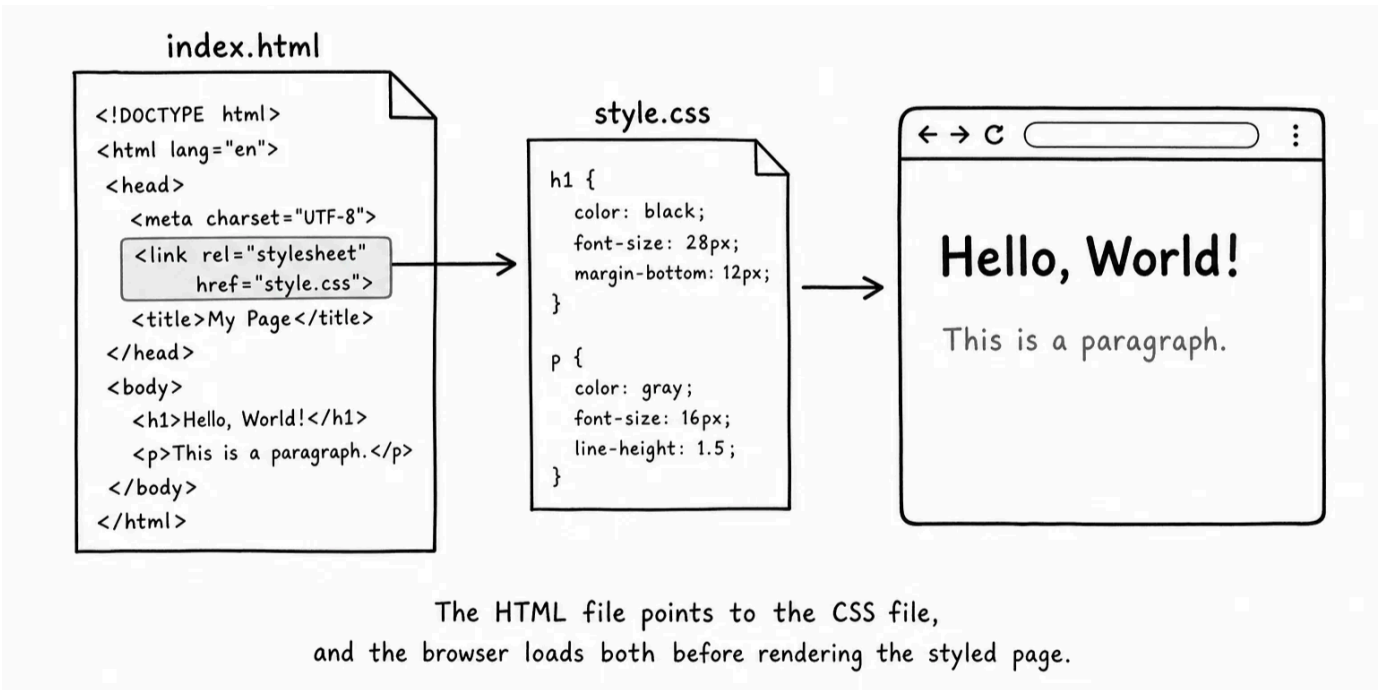
CSS

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f8fafc;
  color: #1f2937;
}

h1 {
  font-size: 40px;
}

p {
  font-size: 18px;
  line-height: 1.6;
}
```

When the browser loads the page, it reads the HTML, finds the stylesheet link, downloads the CSS file, and applies the matching rules to the page.



⚠️ **Check the file path first** - If your CSS does not seem to work at all, check the `href` in your `<link>` element first. A small typo like `styles.css` instead of `style.css` means the browser never loaded the stylesheet.

What CSS can change

CSS can change a lot, but start with the visible stuff: how text reads, how things are spaced, and where they sit.

Text, colors, and backgrounds often show up together:

CSS

```
body {
  color: #111827;
```

```
background-color: #f3f4f6;
}

h1 {
  font-size: 36px;
  font-weight: 700;
  line-height: 1.1;
}
```

Spacing and box styles turn plain content into interface pieces:

```
CSS

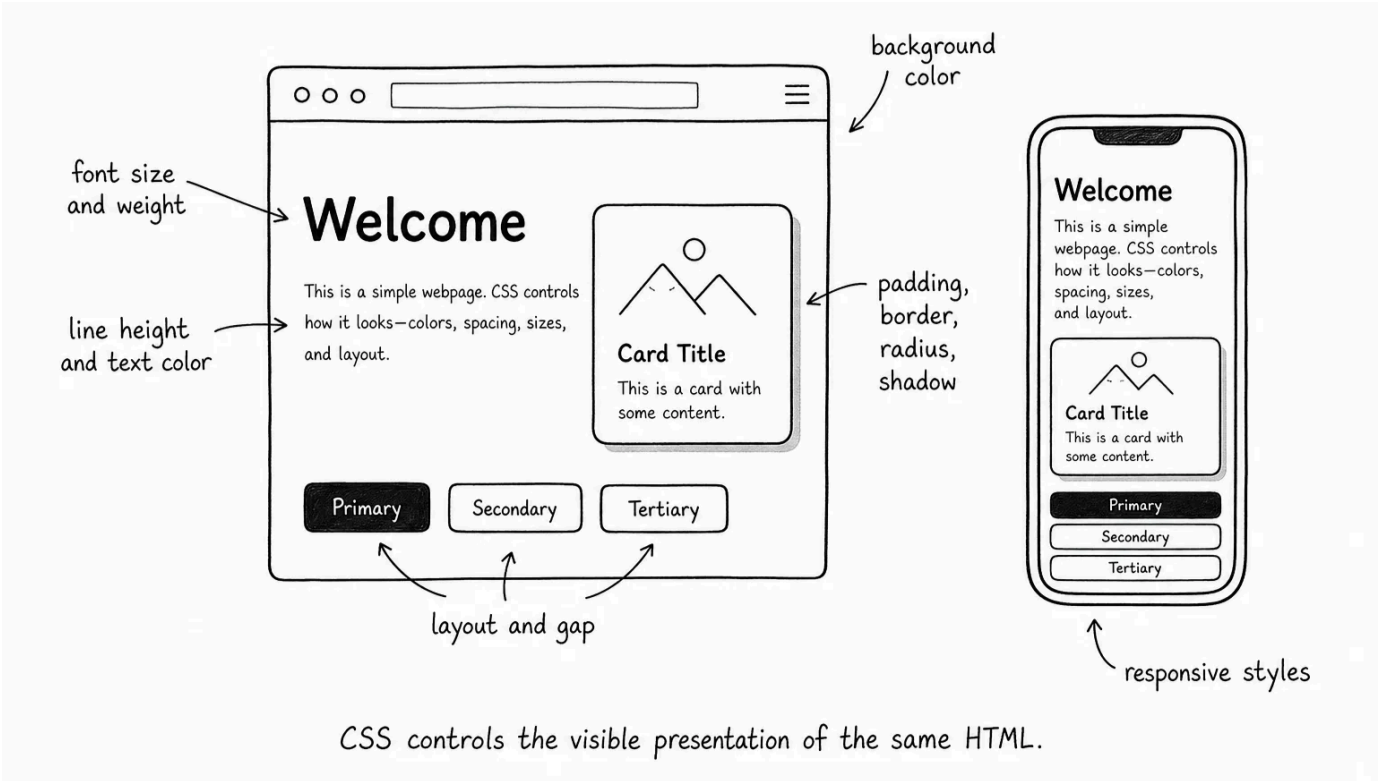
.card {
  padding: 24px;
  margin-bottom: 16px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}
```

Layout and responsive CSS control where things sit, and how that changes on different screens:

```
CSS

.actions {
  display: flex;
  gap: 12px;
}

@media (max-width: 600px) {
  h1 {
    font-size: 28px;
  }
}
```



We will cover each of these properties in the future lessons. For now, notice the shape of the language. CSS is a set of instructions about presentation: make this text bigger, give this box space, put these items in a row, change this layout on small screens.

CSS works with the DOM

In the **HTML pack**, you saw that the browser turns HTML into the DOM, a live tree of elements inside the browser. CSS does not style the raw text file directly. It styles elements in that DOM tree.

If your HTML has this paragraph:

html

```
<p class="lead">CSS makes HTML look designed.</p>
```

then CSS can match it by element name:

CSS

```
p {  
  font-size: 18px;  
}
```

CSS can also match the same paragraph by class:

CSS

```
.lead {  
  color: #4b5563;  
}
```

The browser looks through the DOM, finds the elements that match each selector, and applies the declarations from those rules.

That is why HTML attributes like **class** matter so much. A class gives CSS a clear hook:

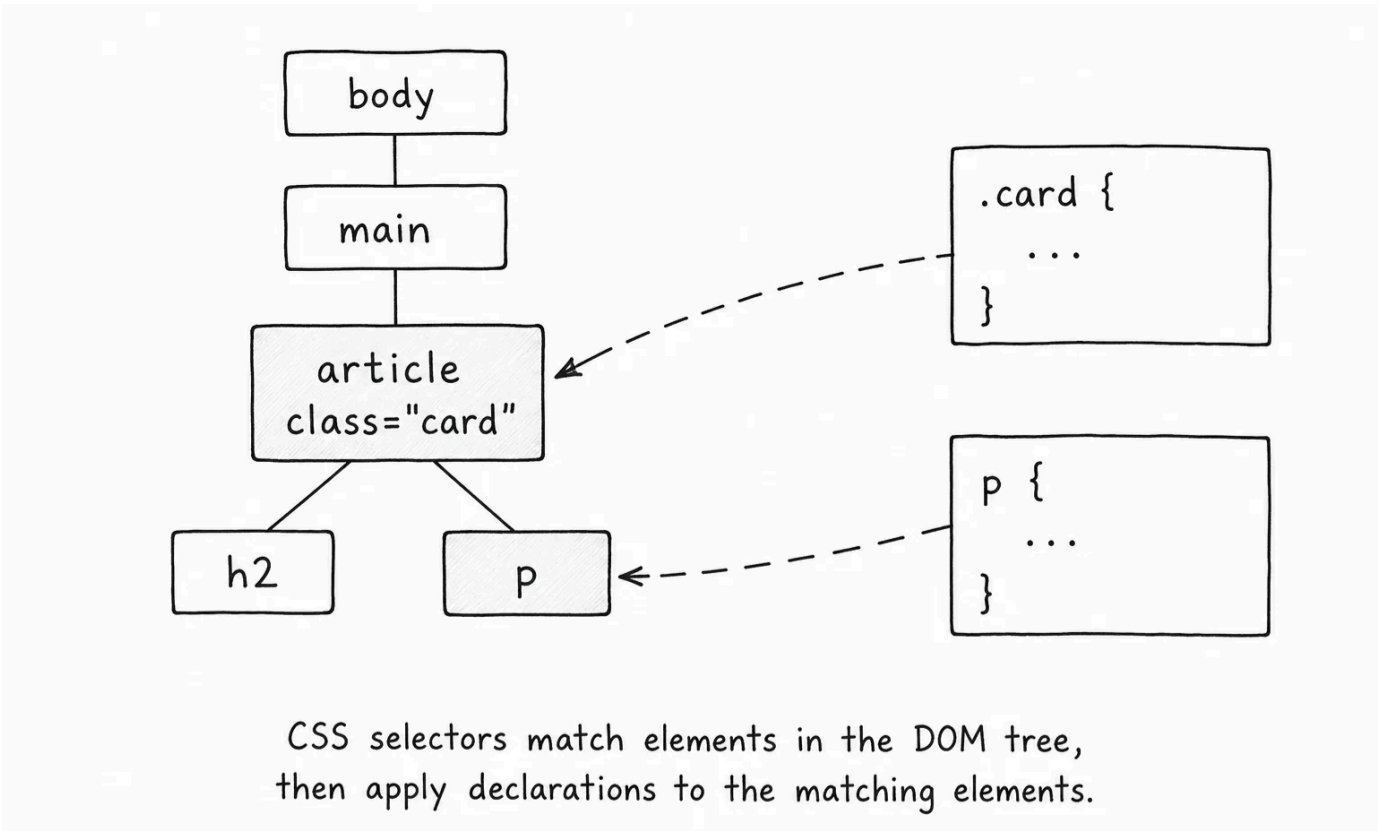
html

```
<article class="card">  
  <h2>Starter plan</h2>  
  <p>Good for small projects.</p>  
</article>
```

CSS

```
.card {  
  background-color: white;  
  padding: 24px;  
  border-radius: 8px;  
}
```

The **article** still means "self-contained content" because of the HTML element. The **card** class says "style this like a card" because of the CSS.

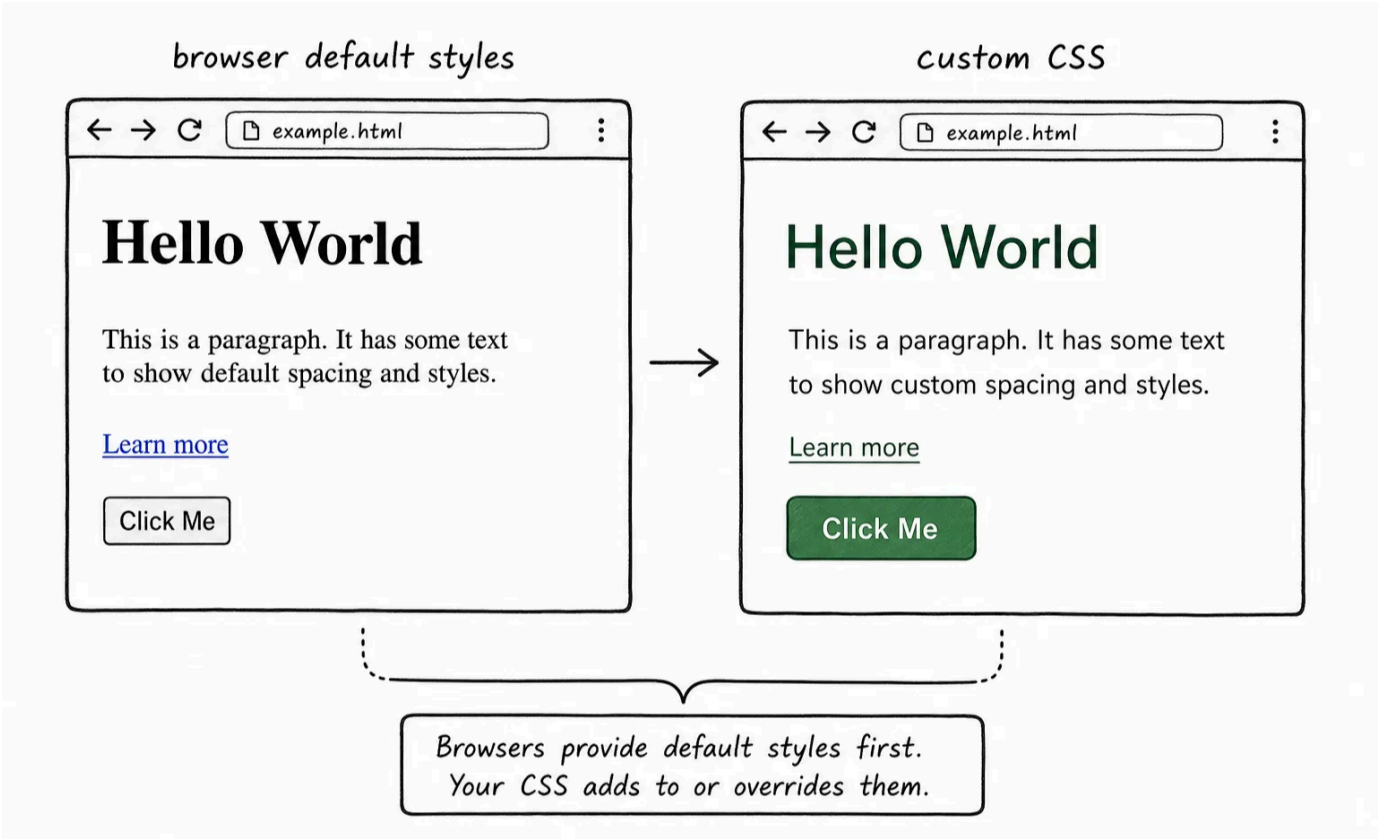


Browsers have default styles

Even before you write any CSS, a page has some style: headings are large and bold, paragraphs have space around them, links are blue and underlined, lists have bullets, and form inputs have borders.

Those styles come from the browser's built-in stylesheet, often called the **user agent stylesheet**. It is the browser's default opinion about how HTML should look.

That is why a plain HTML page is readable without your CSS. The browser is already doing some presentation work, and your CSS adds to or overrides those defaults. You do not need to fight every browser default right away, but you should remember that defaults exist. When something has spacing, color, or a border you did not write, the browser may have supplied it.



💡 **Defaults are useful while learning** - Plain HTML is not supposed to look finished, but it should still be readable. CSS improves the

presentation, but it should not be the only thing making the content understandable.

CSS in real projects

In small projects, you might write one `style.css` file by hand. In larger projects, CSS may appear in different forms:

- A global stylesheet shared by the whole app.
- Component styles that live next to a component.
- Utility classes from a framework like Tailwind CSS.
- CSS variables for shared colors, spacing, and font sizes.
- Styles generated or bundled by a build tool.

The syntax and file structure can change, but the browser still ends up with CSS rules. Those rules still select elements and apply declarations.

This is why CSS basics matter even if you use a framework. A class like `p-4` in Tailwind still means padding. A design token like `--color-primary` still becomes a color value. A component library button still has hover, focus, disabled, spacing, border, and layout rules somewhere underneath.

You do not need to know every CSS architecture on day one. Start with plain CSS, learn how the browser applies it, and the tools will make more sense later.



AI Tutor

Explain why learning plain CSS still matters when a project uses Tailwind, React components, a UI library, or a build tool. Use a button as a simple example, but focus on the mental model: how those tools still become CSS rules that the browser applies to HTML.



Try it

Create a folder with two files: `index.html` and `style.css`.

First, put this in `index.html` without linking the CSS file yet:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>CSS practice</title>
  </head>
  <body>
    <main class="page">
      <h1>CSS practice</h1>
      <p class="lead">This page starts as plain HTML, then CSS changes how it looks.</p>
```



```
<article class="card">
  <h2>Practice card</h2>
  <p>CSS can add space, color, borders, and rounded corners to the same HTML.</p>
  <button>Save changes</button>
</article>
</main>
</body>
</html>
```

Open `index.html` in your browser now. You should see a readable but plain page using the browser's default styles.

Now create `style.css` and add the CSS:

css

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f3f4f6;
  color: #111827;
}

.page {
  max-width: 720px;
  margin: 40px auto;
}

.lead {
  font-size: 20px;
  line-height: 1.5;
  color: #4b5563;
}

.card {
  background-color: white;
  padding: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

button {
  background-color: #2563eb;
  color: white;
  padding: 12px 16px;
  border: 0;
  border-radius: 6px;
}
```

Now add this line inside the `head` of `index.html` , just below the `title` :

html

```
<link rel="stylesheet" href="style.css">
```

Reload the page. The HTML did not change, but the page should look very different because the browser is now loading your CSS.

Try these changes one at a time:

- Change the `body` background color.
- Increase the `.card` padding from `24px` to `40px` .
- Change the button `border-radius` from `6px` to `999px` .

- Open DevTools, click the button, and look at the styles applied to it.

The goal is not to memorize the properties yet. The goal is to see the connection: HTML gives you elements, CSS selects those elements, and declarations change how they appear.

< 0 / 8 >

✔ Knew 0 Items

✦ Learnt 0 Items

⏮ Skipped 0 Items

↺ ResetProgress

Test Your Understanding

What does CSS do?

[Click to Reveal the Answer](#)

✔ Already Know that

✦ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →